MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics/Software Engineering

HEALTHBRIDGE

Community Health Resource Locator

GENERAL PROJECT REPORT

BRIDGE TECH GROUP — GROUP 2

NAME	REGISTRATION NUMBER	ROLES
Tumukunde Isaac	**2024/BCS/162/PS**	**Documentation Leader**
Aryamanya Lucky	**2024/BCS/045/PS**	**Database Designer**
Nambatya Bladina Doreen	**2024/BCS/128/PS**	**Requirements Analyst**
Nanjuki Daphine	**2024/BCS/129/PS**	**Project manager**
Kusingura Jachin	**2024/BCS/091/PS**	**Project tester**
Arinda Leticia	**2024/BCS/043/PS**	**System designer**

1. PROJECT OVERVIEW

HealthBridge is a web-based Community Health Resource Locator developed by Bridge Tech Group as part of the Software Engineering course at Mbarara University of Science and Technology. The system addresses a critical gap in underserved communities, the difficulty of finding accurate, up-to-date information about nearby healthcare services, medicine availability, and health education resources. HealthBridge connects community members to local clinics, pharmacies, and NGO health programmes through an interactive, map-based web interface.

1. Problem Statement

Communities in low-income and rural areas such as those served by Ruharo Mission Hospital and TASO Mbarara lack a centralized, reliable digital resource for locating health services. Clinic hours change without notice, medicine availability is unknown until a patient physically visits a

facility, and health education materials are scattered and inaccessible. This results in wasted journeys, delayed treatment, and poor health outcomes.

2. Proposed Solution

HealthBridge solves this by providing a single, always-available web platform where community members can search for clinics by location and service type, check medicine availability, view results on an interactive map, and read curated health articles. Authorized administrators NGO staff and clinic personnel can update and verify clinic information in real time through a secure admin panel. No app installation is required; the system runs in any browser.

3. Target Organizations

- Non-Governmental Organzations (NGOs) managing community health programmes.
- Community Health Centres for example Ruharo Mission Hospital, TASO Mbarara.
- Low-income community health facilities for example Good Samaritan Clinic.
- Government district health offices seeking centralized resource directories.

4. Functional Requirements (System Behavior)

- The system shall allow users to search for healthcare facilities by entering a location name.
- The system shall display search results on an interactive map with markers.
- The system shall allow users to filter search results by service type e.g. Maternity, Dental
- The system shall allow users to filter search results by medicine availability e.g. Amoxicillin

- The system shall require users to authenticate e.g.log into access the administrative panel.
- The system shall allow authenticated admins to perform Create, Read, Update, and Delete (CRUD) operations on clinic records.
- The system shall allow authenticated admins to perform CRUD operations on medicine inventory for specific clinics.
- The system shall store the history of changes made to a clinic's information for audit purposes.
- The system shall generate a "low update" alert for the head admin if a clinic's record hasn't been modified in 45 days.
- The system shall provide a simple text-based search for users with low bandwidth.
- The system shall display a "verified" badge on clinics whose data has been confirmed in the last 30 days.
- The system shall allow admins to upload and categorize health education articles.
- The system shall allow users to view a list of all health education articles.
- The system shall log all search queries for anonymous analytical purposes (without storing user identities).
- The system shall prevent a user from submitting a review for a clinic without a valid session token (to prevent spam).

10. Non-Functional Requirements (Quality Attributes)

- Performance: The system shall return search results within seconds on a standard broadband connection.
- Usability: A first-time user shall be able to perform a successful clinic search within a few minutes of landing on the homepage, without any training.

- Security: The system shall encrypt all user session data and all data transmitted between the user's browser and the server.
- Security: The system shall hash all administrator passwords using a strong, salted hashing algorithm
- Availability: The system shall have 99.5% uptime during daytime hours (7 AM - 10 PM).
- Scalability: The system shall support users initially, with the architecture allowing for horizontal scaling.
- Reliability: The system shall have a mechanism to automatically rollback a database transaction if it fails, ensuring data integrity.
- Maintainability: The source code shall be well-commented and follow a consistent coding standard to allow a new developer to understand the project within one week.
- Accessibility: The system shall be functional on the latest versions of Chrome, Firefox, and Safari, as well as the default browsers on low-end Android devices.
- Integrity: The system shall validate all user input on the server-side to protect against SQL Injection and Cross-Site Scripting (XSS) attacks.

2. SYSTEM CONTEXT AND PROCESS MODELS

Chosen process model is Agile model which focuses on iterative development, close customer collaboration, and responding to change. Work is done in short cycles called sprints for example 2 weeks.

This is because It allows us to adapt quickly to changing clinic data or user feedback. We can have a product owner e.g. a community health worker to guide us.

1. Context Diagram

The Context Diagram below shows HealthBridge as a single system process and identifies all external entities that exchange data with it. Every actor outside the system boundary either sends data to HealthBridge or receives data from it.

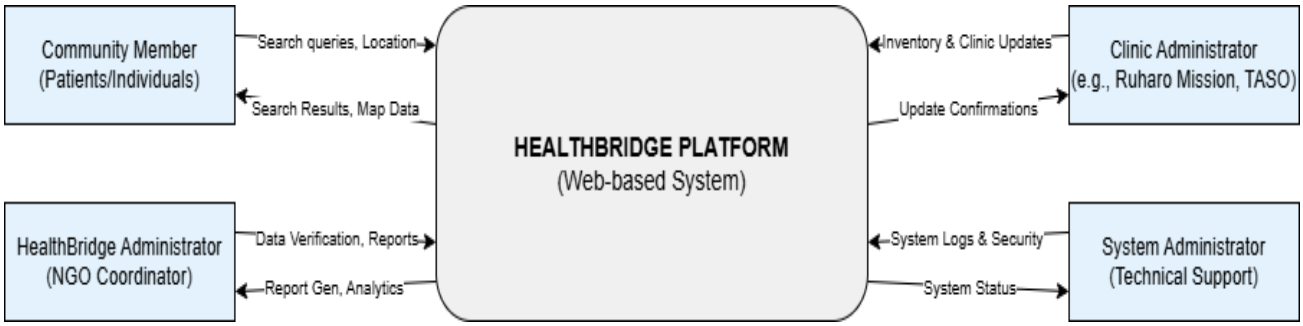


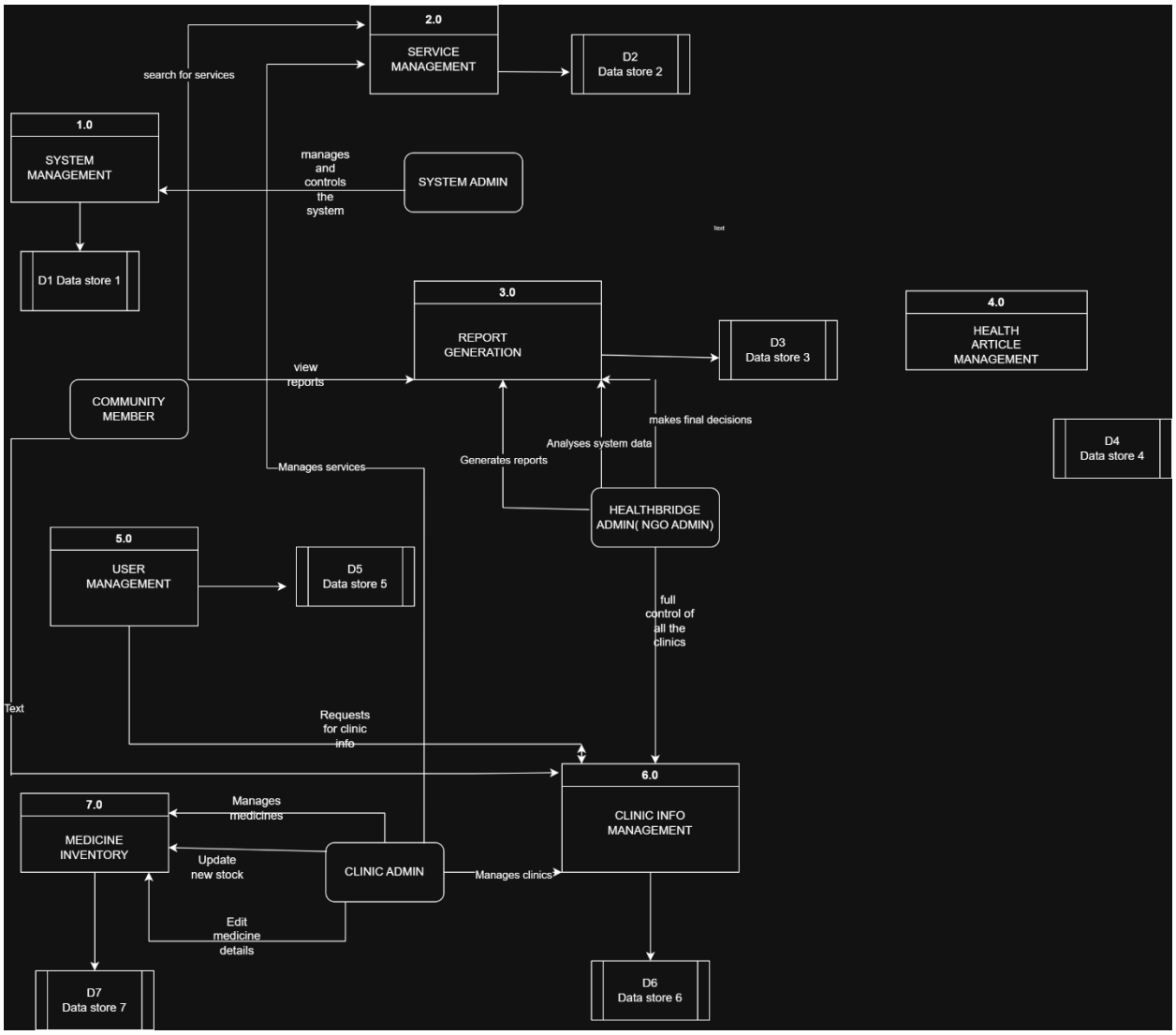
Figure 1: HealthBridge Context Diagram

Five external entities interact with HealthBridge include Community Members (the primary end users who search for clinics and read health articles), Clinic Administrators (who update clinic information and medicine stock), the HealthBridge Administrator (an NGO coordinator who verifies data and manages content), the System Administrator (responsible for backups and security), and a Third-Party Mapping API such as Google Maps or OpenStreetMap (which provides geocoding and map display services).

2. Level 1 Data Flow Diagram — Major Processes

The Level 1 DFD decomposes HealthBridge into eight major processes, each supported by dedicated data stores. Key processes include: User Management (authentication and profiles), Clinic Search and Discovery (location based filtering and result display), Clinic Information

Management (CRUD and audit logging), Medicine Inventory Management (stock updates per clinic), the Map and Geocoding Service (integration with external mapping APIs), Health Article Management; Reporting and Analytics; and System Administration. Six data stores User Database, Clinic Database, Medicine Database, Article Database, Audit Log Database, and Search Log Database support these processes.



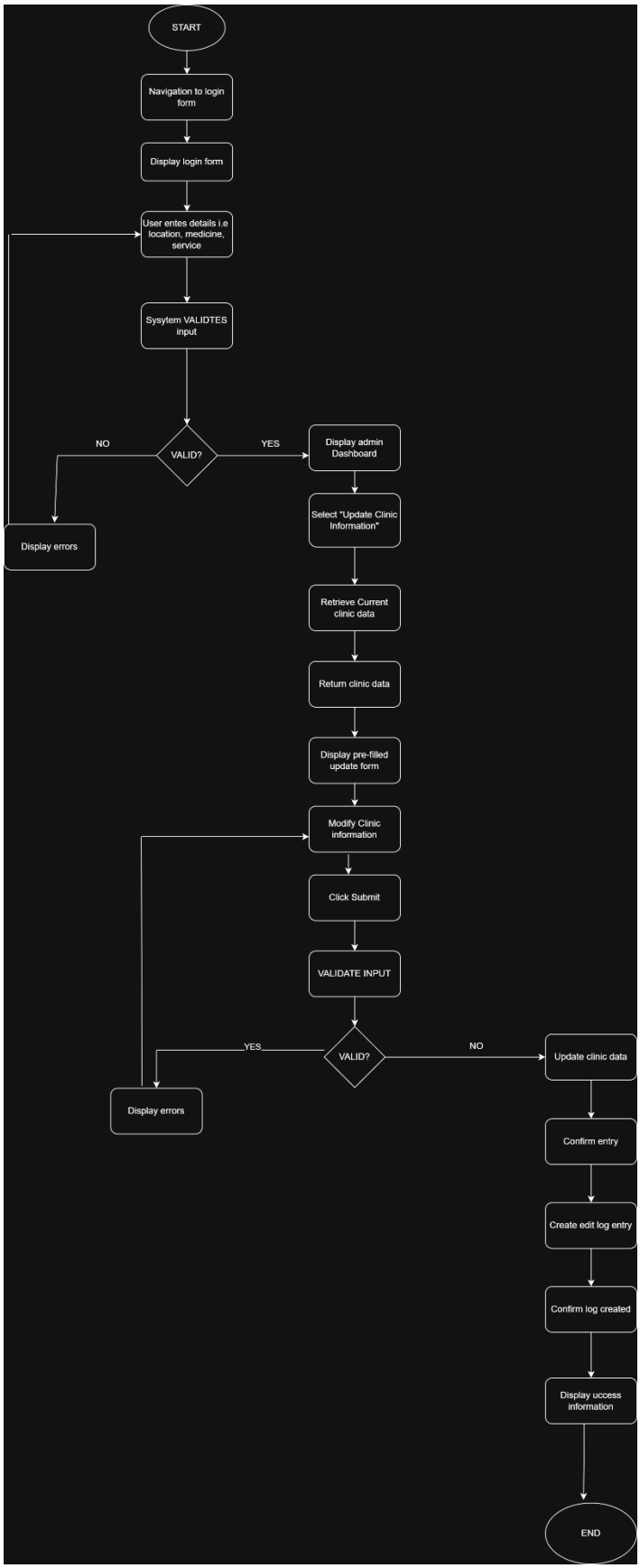
ACTIVITY DIAGRAM

Process Name: Clinic Update by Clinic Admin
Work flow steps
Clinic Administrator navigates to login page

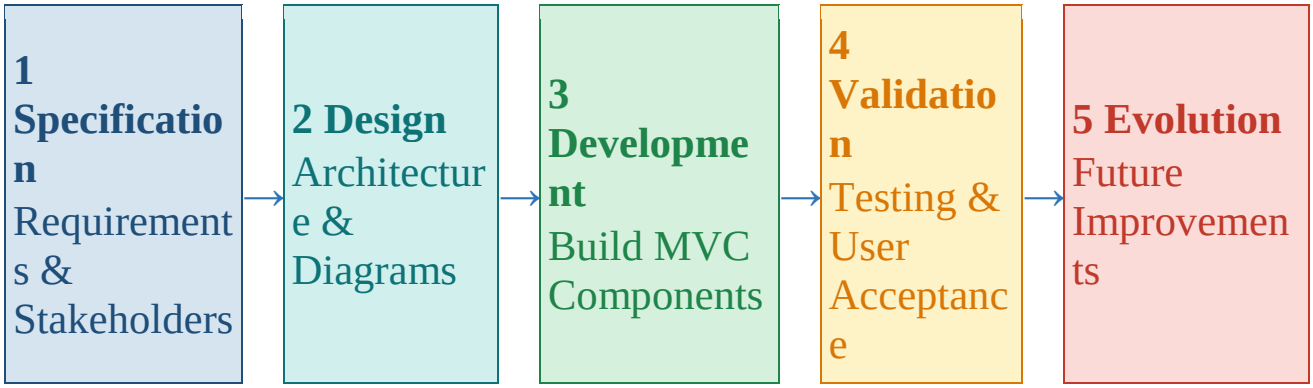
1. System displays login form
2. Admin enters credentials
3. System validates credentials
4. System displays admin dashboard with clinic options
5. Admin selects "Update Clinic Information"
6. System retrieves current clinic data from database
7. System displays pre-filled update form
8. Admin modifies information (hours, phone, services, etc.)
9. Admin submits changes
10. System validates input (format, required fields)
11. System checks for errors

- If errors found: System displays error messages, return to step 8
- If no errors: Proceed to step 13

12. System updates database with new information
13. System creates audit log entry (who changed what and when)
14. System displays success confirmation to admin
15. Process ends


START
Navigation to login form
Display login form
User entes details i.e location, medicine, service
Sysytem VALIDTES input
VALID?
NO
YES
Display errors
Display admin Dashboard
Select "Update Clinic Information"
Retrieve Current clinic data
Return clinic data
Display pre-filled update form
Modify Clinic information
Click Submit
VALIDATE INPUT
VALID?
YES
NO
Display errors
Update clinic data
Confirm entry
Create edit log entry
Confirm log created
Display uccess information
END

3. Software Lifecycle Roadmap



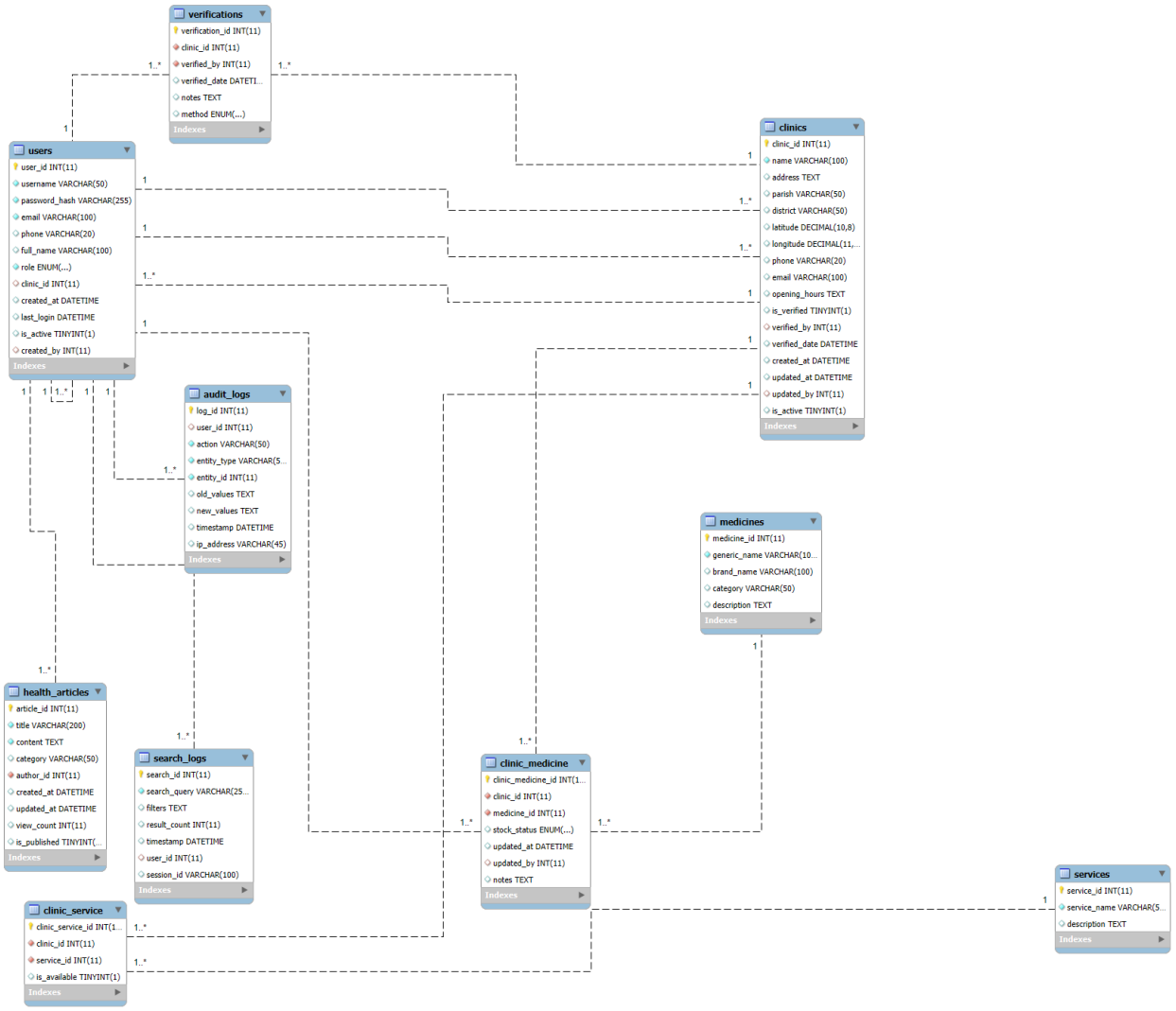
The HealthBridge project follows a structured five-stage software lifecycle.

1) Specification involved interviewing clinic staff, patients, and community leaders to define requirements.
2) Design produced wireframes, database schemas, DFDs, and architecture blueprints. Development is building the MVC-structured application components.
3) Validation tests the system with real users and automated test suites.
4) Evolution plans future improvements including telemedicine integration, an SMS-only interface for feature phone users, and an appointment booking module.

3. DATA MODEL — ENTITY RELATIONSHIP DESIGN

1. Entity Relationship Diagram

The diagram below shows the core entities in the HealthBridge relational database and their relationships. Junction tables (Clinic Service and Clinic Medicine) handle the many-to-many relationships between clinics, services, and medicines.



2. Key Entities and Relationships

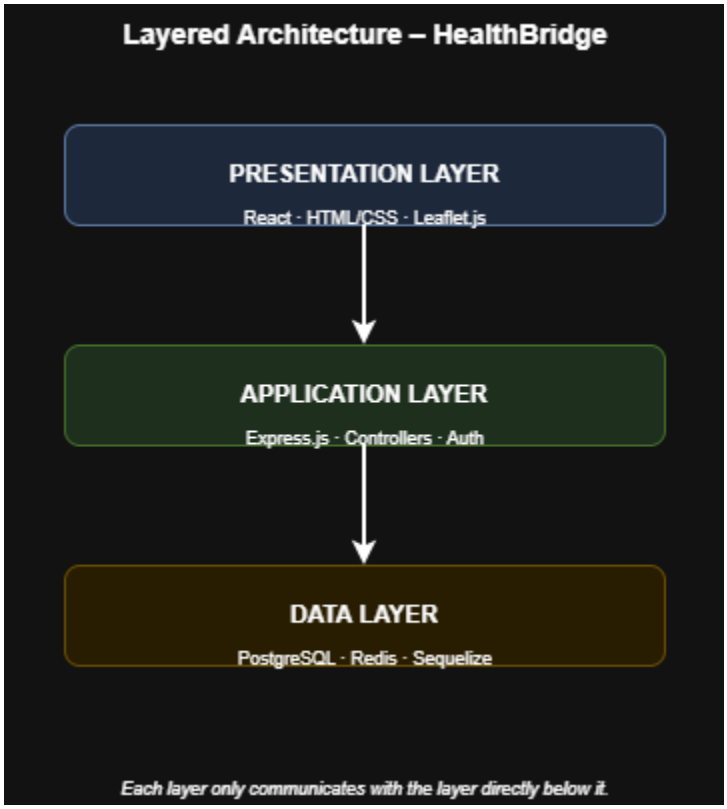
ENTITY 1	RELATIONSHIP	ENTITY 2

users	manages	clinics
clinics	has	Clinic medicine
user	generates	Search logs
Services	Offered by	Clinic service
Clinic service	Belongs to	clinic
medicine	Appears in	Clinic medicine
User	Belongs to	clinic

4. ARCHITECTURAL DESIGN

a. Chosen Pattern: MVC (Model-View-Controller)

HealthBridge adopts the MVC architectural pattern because it is a data-driven web application where the database schema, business logic, and user interface must evolve independently. MVC allows three team members to work on the Model (database), View (frontend), and Controller (backend API) simultaneously without merge conflicts. It aligns naturally with REST API design and is natively supported by the chosen technology stack (Node.js / Express / React).

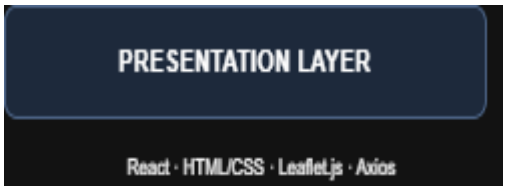


b. Three-Tier Deployment Architecture

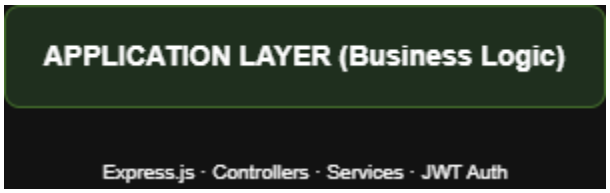
Beyond MVC, the physical deployment follows a three-tier architecture hosted entirely on Google Cloud Platform. The Presentation Layer (React.js frontend) is served via Firebase Hosting with a global CDN. The Application Layer (Node.js/Express) runs on Google Cloud Run, which

auto-scales to handle traffic peaks. The Data Layer uses Cloud SQL (PostgreSQL) for persistent relational data and Cloud Memorystore (Redis) for caching the 50 most frequent clinic search queries, targeting sub-2-second response times even on a 3G connection.

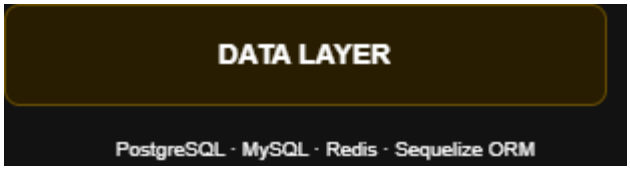
Presentation layers



Application layer



Data layer



c. Non-Functional Requirements

Attribute	Requirement	Implementation
Performance	Search results in under 2 seconds (3G)	Redis caching, SQL indexes on location/service-type, CDN for static assets
Security	Protect health search data and admin credentials	HTTPS/TLS, bcrypt+salt, JWT (24h expiry), parameterised SQL, no sensitive search logging
Availability	99% uptime, auto-recovery from failure	Google Cloud Run auto-restart, daily PostgreSQL backups, admin panel for non-technical updates

Attribute	Requirement	Implementation
Maintainability	Non-developers can update clinic data	Admin CRUD panel, MVC separation, clear Git branching, documented API
Scalability	City → nationwide without restructuring	Stateless REST API, horizontal scaling on Cloud Run, Redis reduces DB load 60-80%
Privacy	GDPR/HIPAA principles for health searches	Sensitive search terms (HIV, mental health) never written to server logs

5. SYSTEM FEATURES AND USE CASES

1. Actor Summary

ACTOR	DESCRIPTION
Community member	A patient or community member seeking health services. They are the primary end-users of the system.
Clinic administrator	Staff member at a specific clinic responsible for keeping that clinic's data accurate and up-to-date.
HealthBridge Administrator	Coordinator at the NGO who oversees the entire system, verifies data quality, and manages content
System administrator	Technical person responsible for system maintenance, backups, security, and performance
Third party mapping API	External system actor that provides geocoding and map visualization services.

6. SOFTWARE TESTING REPORT SUMMARY

Reviewed and summarised from the counter-group testing report submitted for HealthBridge.

1. Testing Overview

An independent counter-group conducted structured software testing on the HealthBridge system covering all major functional areas. The testing aimed to evaluate quality, reliability, security, and performance before deployment. A combination of manual and automated testing was employed, applying both black-box (functional validation) and white-box (code-level validation) techniques. Testing progressed through three levels: Unit → Integration → System.

Testing Feedback (from Counterpart Group)

The following comments were collected during peer testing;

- Slow response under heavy load, The system showed delays when multiple users accessed it simultaneously.
- SQL injection vulnerability, Input fields were not fully protected against malicious SQL attacks.
- Outdated clinic data displayed, Some clinic information shown to users was not current.
- Map fails on low internet, The interactive map did not load properly on weak connections.
- Broken links not handled, Invalid links caused error pages instead of user-friendly messages.
- Pharmacy search returned wrong results, Searching for pharmacies showed other facility types.
- Empty search not properly validated, Blank searches did not always prompt the user to enter data.
- Invalid credentials sometimes granted access, Login with wrong passwords occasionally worked.
- Fast multiple searches caused delays, Rapid consecutive searches slowed down the system.

Challenges Encountered During Testing

- Difficulty simulating real-time data updates in a test environment.
- Limited access to real community users for User Acceptance Testing (UAT).

- API dependency failures — mapping services occasionally unavailable during tests.
- Security vulnerabilities (SQL injection) discovered late in the testing cycle.
- Performance issues under concurrent load not detected until system-level tests.

7. RECOMMENDATIONS AND NEXT STEPS

High Priority Improvements

- Deploy Redis caching layer for the 50 most frequent clinic searches to address performance under concurrent load.
- Enable horizontal auto-scaling on Google Cloud Run and add a load balancer to distribute traffic.
- Implement the is_verified display flag on the frontend to clearly indicate unverified clinic data.
- Add an offline detection handler and graceful fallback UI when the mapping service is unavailable.

Medium-Term Recommendations

- Conduct penetration testing and hire or consult a security specialist for a formal audit.
- Run regression testing after every fix to ensure new changes do not re-introduce resolved defects.
- Recruit real community members at a local health centre for formal User Acceptance Testing (UAT).
- Establish a monthly data verification schedule and build admin-facing tools to flag stale clinic records.

Long-Term Evolution

- Offline-First PWA: Implement service workers so the app works in low-connectivity rural areas a cached version of nearby clinics would be accessible without internet.
- SMS-Only Interface: Build an SMS bot via Africa's Talking so users with feature phones can text a keyword and receive the nearest clinic by SMS.
- Appointment Booking Module: Allow community members to book appointments directly through HealthBridge, reducing queue times and improving health facility planning.

- Community Verification: Implement a crowd-sourced flagging system so community members can report outdated data supplementing the admin verification team.

DEPLOYMENT PLAN (Production)

1. **Environment**

Component	Technology	Hosting
Frontend	React.js (build)	Firebase Hosting / Vercel
Backend	Node.js + Express	Google Cloud Run (container)
Database	PostgreSQL	Google Cloud SQL
Cache	Redis	Google Cloud Memorystore
Static assets	Images, CSS	Cloud CDN
Domain	SSL (HTTPS)	Custom domain (for example healthbridge.ug)

2. **Deployment Steps (Future Work)**
   a. Set up PostgreSQL schema (from schema.sql)
   b. Deploy backend to Cloud Run with environment variables
   c. Deploy frontend build to Firebase Hosting
   d. Configure custom domain and SSL certificate
   e. Set up automated backups (daily)
   f. Implement monitoring (uptime, error alerts)

2. **SAMPLE CLINICS INCLUDED**

Clinic Name	District	Services
Ruharo Mission Hospital	Mbarara	Maternity, General Consultation,

		Pharmacy, HIV/AIDS
TASO Mbarara	Mbarara	HIV/AIDS, Pharmacy, Counselling
Kampala General Hospital	Kampala	Maternity, Dental, Emergency
Nsambya Hospital	Kampala	Maternity, Paediatrics
Gulu Regional Hospital	Gulu	General Consultation, Surgery, HIV/AIDS
Jinja Regional Hospital	Jinja	Maternity, General Consultation
Mental Health Center – Butabika	Kampala	Mental Health, Counselling

CONCLUSION

HealthBridge successfully demonstrates how a well-designed, privacy-aware web application can solve the real-world problem of accessing reliable health resource information in underserved Ugandan communities. The prototype validates all core user journeys, from community member search to clinic admin updates to super admin verification while upholding strong privacy principles for sensitive health queries.

The architectural design (MVC + layered, 4+1 views, client-server) is sound and ready for implementation using the documented technology stack (React, Node.js, PostgreSQL, Redis). Feedback from peer testing has been incorporated into the future enhancements list, ensuring that the next iteration will be even more robust, accessible, and scalable.

HealthBridge is not just a student project it but rather has the potential to be deployed in real NGOs and health centres, improving health outcomes for thousands of Ugandans.